

Assignment 8: Time Series Analysis

Sean Burke

Fall 2025

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on generalized linear models.

Directions

1. Rename this file `<FirstLast>_A08_TimeSeries.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up

1. Set up your session:
 - Check your working directory
 - Load the tidyverse, lubridate, zoo, and trend packages
 - Set your ggplot theme

```
getwd()
```

```
## [1] "/home/guest/EDE_Fall2025"
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(lubridate)
library(zoo)
```

```
##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##      as.Date, as.Date.numeric
```

```
library(here)
```

```
## here() starts at /home/guest/EDE_Fall2025
```

```
library(ggthemes)

myTheme1 <- theme_base() +
  theme(
    plot.background = element_rect(
      color = 'black',
      fill = 'white'
    ),
    plot.title = element_text(
      color = 'black'
    ),
    axis.text = element_text(
      color = 'black'
    ),
    axis.ticks = element_line(
      color = 'purple',
      linewidth = 1
    )
  )
```

2. Import the ten datasets from the Ozone_TimeSeries folder in the Raw data folder. These contain ozone concentrations at Garinger High School in North Carolina from 2010-2019 (the EPA air database only allows downloads for one year at a time). Import these either individually or in bulk and then combine them into a single dataframe named **GaringerOzone** of 3589 observation and 20 variables.

```
#1
```

```
Grainger1 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2010_raw.csv"))
Grainger2 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2011_raw.csv"))
Grainger3 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2012_raw.csv"))
Grainger4 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2013_raw.csv"))
Grainger5 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2014_raw.csv"))
Grainger6 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2015_raw.csv"))
Grainger7 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2016_raw.csv"))
Grainger8 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2017_raw.csv"))
Grainger9 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2018_raw.csv"))
Grainger10 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_03_GaringerNC2019_raw.csv"))
```

```
Garinger_Merged <- rbind(Grainger1, Grainger2, Grainger3, Grainger4, Grainger5, Grainger6, Grainger7, G
```

Wrangle

3. Set your date column as a date class.
4. Wrangle your dataset so that it only contains the columns Date, Daily.Max.8.hour.Ozone.Concentration, and DAILY_AQI_VALUE.
5. Notice there are a few days in each year that are missing ozone concentrations. We want to generate a daily dataset, so we will need to fill in any missing days with NA. Create a new data frame that contains a sequence of dates from 2010-01-01 to 2019-12-31 (hint: `as.data.frame(seq())`). Call this new data frame Days. Rename the column name in Days to “Date”.
6. Use a `left_join` to combine the data frames. Specify the correct order of data frames within this function so that the final dimensions are 3652 rows and 3 columns. Call your combined data frame GaringerOzone.

```
# 3
Garinger_Merged$Date <- as.Date(Garinger_Merged$Date, format = "%m/%d/%Y")

class(Garinger_Merged$Date)

## [1] "Date"

# 4
Garinger_Merged <- select(Garinger_Merged, Date, Daily.Max.8.hour.Ozone.Concentration, DAILY_AQI_VALUE)

# 5
Days <- as.data.frame(seq(from = as.Date("2010/01/01"), to = as.Date("2019/12/31"), by = "day"))
colnames(Days) <- ("Date")

# 6
GaringerOzone <- left_join(Days, Garinger_Merged)

## Joining with 'by = join_by(Date)'
```

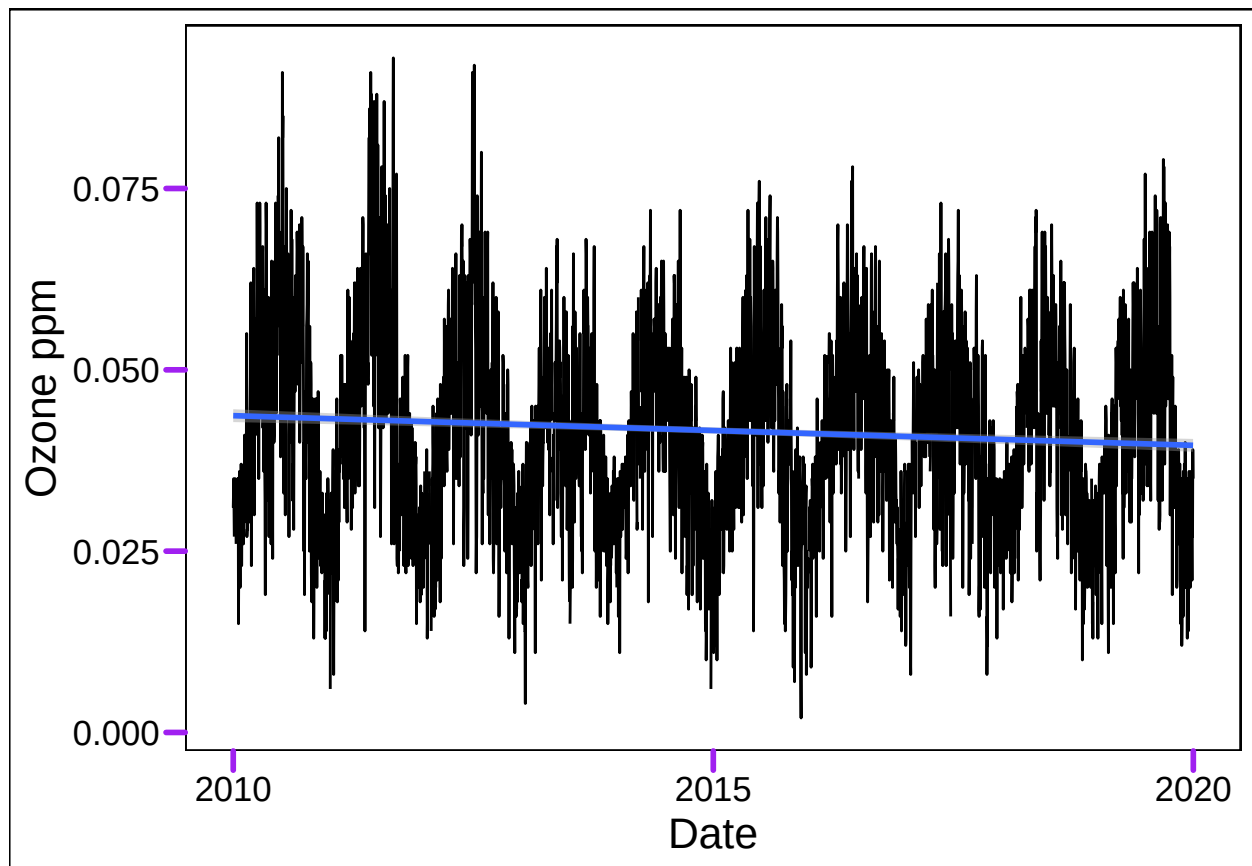
Visualize

7. Create a line plot depicting ozone concentrations over time. In this case, we will plot actual concentrations in ppm, not AQI values. Format your axes accordingly. Add a smoothed line showing any linear trend of your data. Does your plot suggest a trend in ozone concentration over time?

```
#7
ggplot(GaringerOzone, aes(x = Date,
                          y = Daily.Max.8.hour.Ozone.Concentration)) +
  geom_line() +
  ylab("Ozone ppm") +
  geom_smooth(method = lm) +
  myTheme1

## 'geom_smooth()' using formula = 'y ~ x'

## Warning: Removed 63 rows containing non-finite outside the scale range
## ('stat_smooth()').
```



Answer: Our plot suggests that there is a trend in ozone concentration over time. It seems that over time ozone concentrations have been slowly falling, not by a large amount but it appears to have dropped by .008 over 10 years.

Time Series Analysis

Study question: Have ozone concentrations changed over the 2010s at this station?

8. Use a linear interpolation to fill in missing daily data for ozone concentration. Why didn't we use a piecewise constant or spline interpolation?

#8

```
GaringerOzone_Interpolated <- GaringerOzone %>%
  mutate(Daily.Max.8.hour.Ozone.Concentration = zoo::na.approx(Daily.Max.8.hour.Ozone.Concentration)) %>%
  mutate(DAILY_AQI_VALUE = zoo::na.approx(DAILY_AQI_VALUE))
```

Answer: We don't want to use piecewise constant because the nearest dates, both earlier and later, are equal distant to the missing data so there is no closest value to set the missing data to. We do not want to use spline because setting up a quadratic function to fill in data with values on either side would be too complicated and the solution does not require a overengineered solution.

9. Create a new data frame called `GaringerOzone.monthly` that contains aggregated data: mean ozone concentrations for each month. In your pipe, you will need to first add columns for year and month to form the groupings. In a separate line of code, create a new Date column with each month-year combination being set as the first day of the month (this is for graphing purposes only)

```
#9
GaringerOzone.monthly <- GaringerOzone_Interpolated %>%
  mutate(Year = year(Date), Month = month(Date)) %>%
  group_by(Year, Month) %>%
  summarise(mean_ozone = mean(Daily.Max.8.hour.Ozone.Concentration)) %>%
  mutate(Date = my(paste0(Month, "-", Year)))
```

'summarise()' has grouped output by 'Year'. You can override using the
'.groups' argument.

10. Generate two time series objects. Name the first `GaringerOzone.daily.ts` and base it on the dataframe of daily observations. Name the second `GaringerOzone.monthly.ts` and base it on the monthly average ozone values. Be sure that each specifies the correct start and end dates and the frequency of the time series.

```
#10
GaringerOzone.daily.ts <- ts(GaringerOzone_Interpolated$Daily.Max.8.hour.Ozone.Concentration,
  start = c(2010, 1),
  frequency = 365)

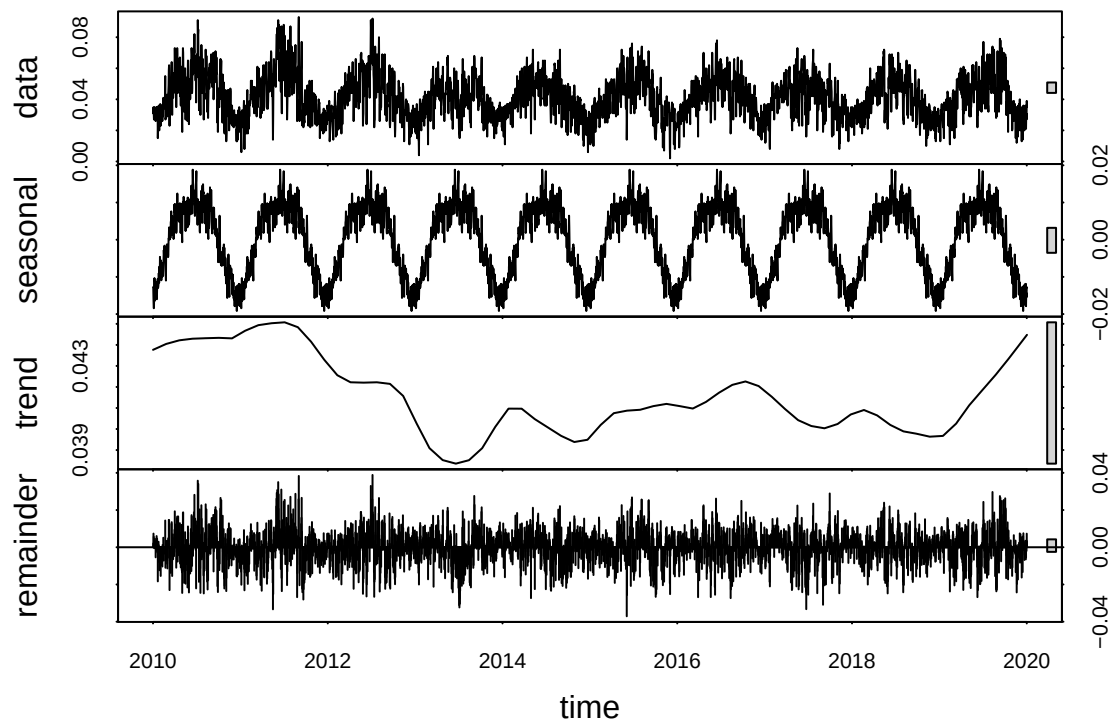
GaringerOzone.monthly.ts <- ts(GaringerOzone.monthly$mean_ozone,
  start = c(2010,1),
  frequency = 12)
```

11. Decompose the daily and the monthly time series objects and plot the components using the `plot()` function.

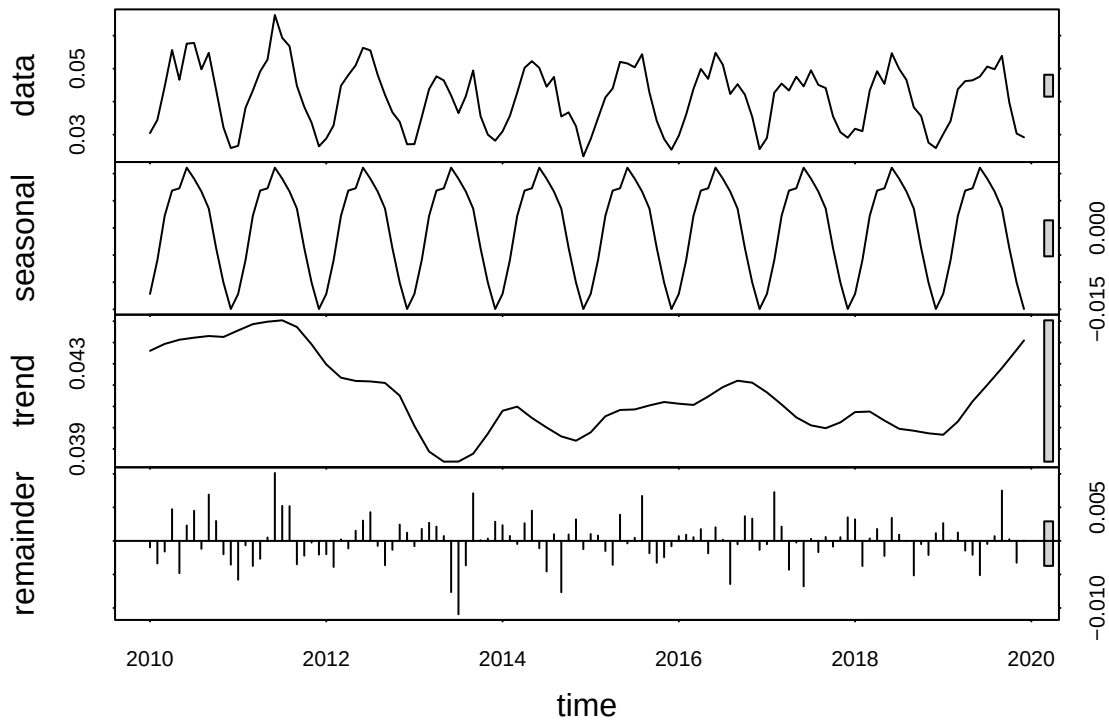
```
#11
GaringerOzoneDaily_Decomposed <- stl(GaringerOzone.daily.ts, s.window = "periodic")

GaringerOzoneMonthly_Decomposed <- stl(GaringerOzone.monthly.ts, s.window = "periodic")

plot(GaringerOzoneDaily_Decomposed)
```



```
plot(Garinger0zoneMonthly_Decomposed)
```



12. Run a monotonic trend analysis for the monthly Ozone series. In this case the seasonal Mann-Kendall is most appropriate; why is this?

```
#12
MonthlyOzoneTrend <- trend::smk.test(GaringerOzone.monthly.ts)
summary(MonthlyOzoneTrend)
```

```
##
## Seasonal Mann-Kendall trend test (Hirsch-Slack test)
##
## data: GaringerOzone.monthly.ts
## alternative hypothesis: two.sided
##
## Statistics for individual seasons
##
## H0
##
```

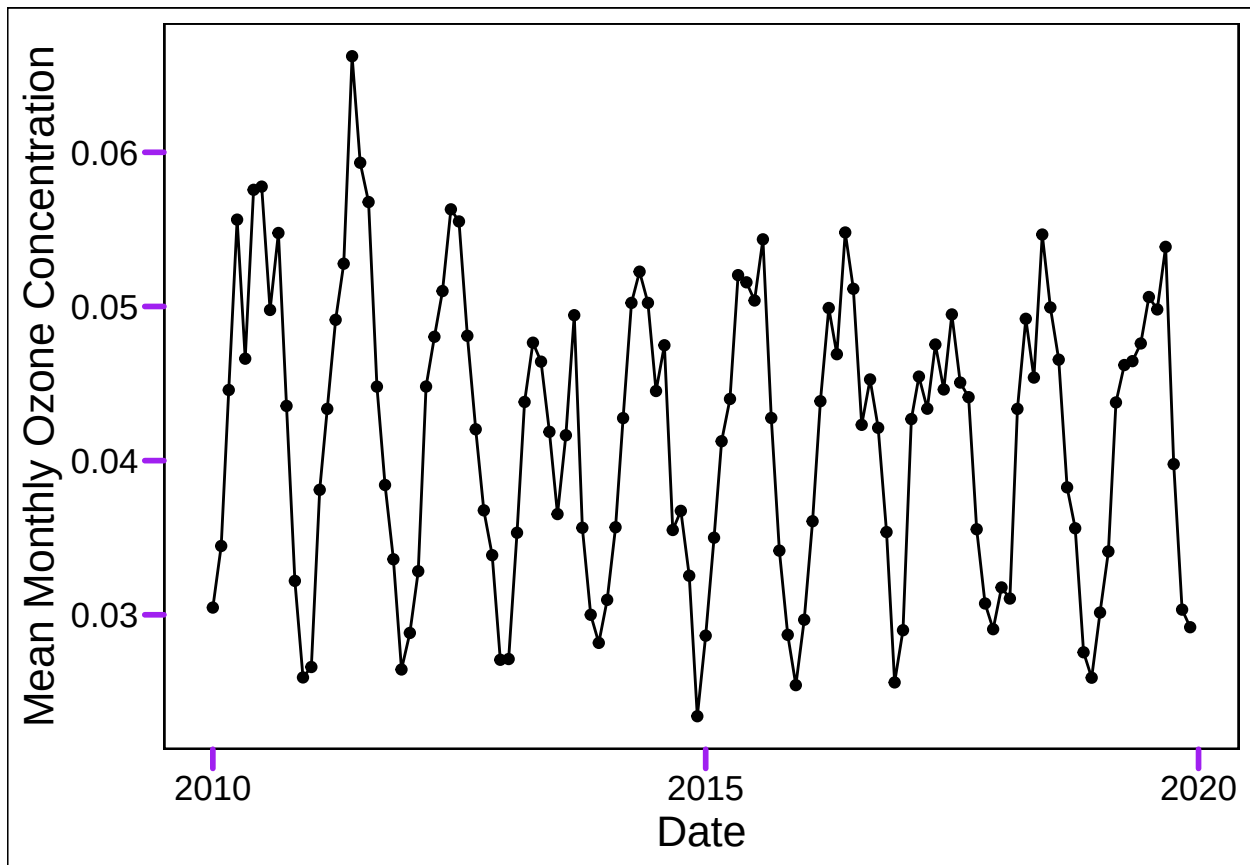
	S	varS	tau	z	Pr(> z)	
## Season 1:	S = 0	15	125	0.333	1.252	0.21050
## Season 2:	S = 0	-1	125	-0.022	0.000	1.00000
## Season 3:	S = 0	-4	124	-0.090	-0.269	0.78762
## Season 4:	S = 0	-17	125	-0.378	-1.431	0.15241
## Season 5:	S = 0	-15	125	-0.333	-1.252	0.21050
## Season 6:	S = 0	-17	125	-0.378	-1.431	0.15241
## Season 7:	S = 0	-11	125	-0.244	-0.894	0.37109
## Season 8:	S = 0	-7	125	-0.156	-0.537	0.59151

```
## Season 9:   S = 0   -5  125 -0.111 -0.358  0.72051
## Season 10:  S = 0 -13  125 -0.289 -1.073  0.28313
## Season 11:  S = 0 -13  125 -0.289 -1.073  0.28313
## Season 12:  S = 0  11  125  0.244  0.894  0.37109
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Answer: Seasonal Mann-Kendall is the most appropriate monotonic trend analysis because our data is seasonal, non-linear, non-parametric, and does not have any missing data.

13. Create a plot depicting mean monthly ozone concentrations over time, with both a `geom_point` and a `geom_line` layer. Edit your axis labels accordingly.

```
# 13
ggplot(GaringerOzone.monthly, aes(
  x = Date,
  y = mean_ozone)) +
  geom_point() +
  geom_line() +
  ylab("Mean Monthly Ozone Concentration") +
  myTheme1
```



14. To accompany your graph, summarize your results in context of the research question. Include output from the statistical test in parentheses at the end of your sentence. Feel free to use multiple sentences in your interpretation.

Answer: From our graph, we can see that average ozone levels have slightly decreased over the 2010s. From our seasonal Mann-Kendall test we can see that the coefficients S and tau also supports this by being negative every year except the first and last years which indicates an overall decreasing trend (S = 15, -1, -4, -17, -15, -17, -11, -7, -5, -13, -13, 11) and (tau = .333, -.022, -.269, -1.431, -1.252, -1.431, -.894, -.537, -.358, -1.073, -1.073, .894).

15. Subtract the seasonal component from the `GaringerOzone.monthly.ts`. Hint: Look at how we extracted the series components for the `EnoDischarge` on the lesson Rmd file.
16. Run the Mann Kendall test on the non-seasonal Ozone monthly series. Compare the results with the ones obtained with the Seasonal Mann Kendall on the complete series.

#15

```
GaringerMonthlyNonSeasonal <- GaringerOzone.monthly.ts - GaringerOzoneMonthly_Decomposed$time.series[,1]
```

#16

```
NonseasonalMonthlyOzoneTrend <- trend::mk.test(GaringerMonthlyNonSeasonal)
```

```
NonseasonalMonthlyOzoneTrend
```

```
##
## Mann-Kendall trend test
##
## data: GaringerMonthlyNonSeasonal
## z = -2.672, n = 120, p-value = 0.00754
## alternative hypothesis: true S is not equal to 0
## sample estimates:
##          S          varS          tau
## -1.179000e+03  1.943657e+05 -1.651376e-01
```

```
MonthlyOzoneTrend
```

```
##
## Seasonal Mann-Kendall trend test (Hirsch-Slack test)
##
## data: GaringerOzone.monthly.ts
## z = -1.963, p-value = 0.04965
## alternative hypothesis: true S is not equal to 0
## sample estimates:
##      S varS
## -77 1499
```

Answer: It would seem that the non-seasonal test performed better than the seasonal test, seeing as the non-seasonal test had a much lower p-value than the seasonal test. The non-seasonal test also has a lower z score than the seasonal test but less S and var S which also all indicate a better fit. So seasonality provides more variability in our test but makes it not as practical as our non-seasonal test.